LOAP AIR Designs

November 9, 2026

Contents

1 General Terms & Definitions **2**

2 SHA-256 AIR **3**

Chapter 1

General Terms & Definitions

Atomic AIR Matrix. In general Computer Science, atomicity is a property denoting the indivisibility of an operation (i.e., an atomic operation runs to completion). In the context of AIR matrices, we use the term *atomic AIR matrix* to refer to an AIR matrix that is standalone (i.e., it does not embed other AIR matrices).

Composite AIR Matrix. A *composite AIR matrix* is an AIR matrix that embeds other (smaller) AIR matrices in their entirety. This means that each sub-AIR matrix is governed by its own constraints, in addition the constraints imposed by the composite matrix.

Register. By *register* we understand a column in the AIR matrix. In the context of composite AIR matrices we identify two classes of registers:

1. **Preserved Registers** are registers who cannot be repurposed and allocated to other AIR matrices.
2. **Temporary Registers** are registers who are free to be repurposed and allocated to other AIR matrices as needed.

For each register, we might specify a *bit width*. Note that in AIR matrices all values are elements from a finite field $\mathbb{F}_p$ (whose bit-width will generally be larger than our specification). As such, the bit width that we specify is a logical limit of the register (not a strict one). For example a 32-bit register should only store values that can be represented in 32 bits, otherwise some AIR constraint will fail; likewise, if we do not specify a bit-width, the register can safely store any arbitrary field element.

Chapter 2

SHA-256 AIR

Without loss of generality, `LOAP` focuses exclusively on the SHA family of hashes, in particular on the SHA-256 algorithm as described in [Nat15]. The encoding of the hash computation is realized as an atomic AIR matrix of dimension $Rows \times Cols = (177B+1) \times 291$, where B is the number of 512-bit blocks of the message including padding. Semantically, each register serves the following purpose:

- Num_Rows is a register storing the index of each trace row. It follows that the initial value of this register is 0, and the last value will be $Rows$ (the total height of the matrix).

  Remark 1. *Since the height of the trace is dynamic in the user input, it must be communicated to the verifier in some form. For simplicity, we assume that the height is included in the messages accompanying each protocol exchange.*

- Quotient, Remainder are two registers that store the quotient and remainder of the division between the number of rows and 177.

- Inverse is the multiplicative inverse of Remainder $-$ 176.

- Control is a group of 2 single-bit registers that select which constraints are enforced for the current row.

- W_Selector is a group of 17 single-bit registers that denote which schedule word is used in the current round. The registers slide right after every round until they reach the last position, and remain fixed afterwards. The last two selectors point to the same W register, while the last register serves as an indicator that the first 16 rounds have been completed.

- Constant is a single 32-bit register storing the current round constant.

- W is a group of 16 32-bit registers, encoding 16 of the 64 32-bit words of the SHA-256 message schedule. These registers will act as a rolling window, with W[0] being the oldest word and W[15] being the most recent.

- a...h, $\mathsf{T}_1, \mathsf{T}_2$ are 10 32-bit registers corresponding to the working variables of the SHA-256 compression function. At each step, these registers store the same values as the variables with identical names in the specification [Nat15].

- Bits is a group of 224 single-bit registers, split as 7 sets of 32 register each. Naturally, each set encodes the bitwise decomposition of a 32-bit number. The registers serve in aiding with enforcing the bitwise computations required by the compression function.
- Bitwise_Output is a group of two 32-bit registers storing the outputs of the bitwise functions $\sum, \sigma, Ch, Maj$ (see [Nat15] for more details). At different rows in the AIR matrix, the registers will store the outputs of different functions.
- Carry is a group of 8 single-bit registers serving as an aid for modular reduction. In particular for large sums, where the end result is almost always guaranteed to exceed 32 bits, the Carry register is used to also reduce it modulo 2^{32}.
- Hash is a group of 8 32-bit registers storing the current hash. At the end of the 64 rounds, the registers store the final processed hash, and at the beginning of the rounds it stores the input hash respectively.

Conceptually, each SHA round is represented by 2 trace rows for the first 16 rounds. For subsequent rounds, we employ 3 rows for the encoding: the first row serves as the computation of the next schedule word, while the remaining two encode half a compression step each. The final row after the last round serves as an alignment step, ensuring the output hash for the current block becomes the input hash for the next one. Thus, we encode an the process of compressing an entire block in 177 rows: 176 rows for the compression function, 1 row for input / output.

To enforce that valid traces respect the previous semantics, we employ constraints split into 4 categories: *general constraints* governing the entire trace, *schedule constraints* governing the correct computation of the next message schedule words, *round constraints* governing the correct execution of the rest of the compression step, and *state machine constraints* enforcing the correctness of the Control and W_Selector bits.

General Constraints

1. The number of rows increases by 1 at every step.

$$\mathsf{Num_Rows}(gx) - \mathsf{Num_Rows}(x) - 1 = 0$$

for $1 \leq x \leq g^{177B-1}$.

2. The Control, W_Selector, Carry and Bits registers hold only single-bit values.

$$\begin{aligned}
\mathsf{Control}[i](x) * (1 - \mathsf{Control}[i](x)) = 0 \quad & \forall i \in \{0, 1\} \\
\mathsf{W_Selector}[i](x) * (1 - \mathsf{W_Selector}[i](x)) = 0 \quad & \forall i \in \{0, ..., 16\} \\
\mathsf{Carry}[i](x) * (1 - \mathsf{Carry}[i](x)) = 0 \quad & \forall i \in \{0, ..., 7\} \\
\mathsf{Bits}[i, j](x) * (1 - \mathsf{Bits}[i, j](x)) = 0 \quad & \forall i \in \{0, ..., 6\}, j \in \{0, ..., 31\}
\end{aligned}$$

for $1 \leq x \leq g^{177B}$.

3. The Quotient and Remainder encode a correct division.

$$177 * \mathsf{Quotient}(x) + \mathsf{Remainder}(x) - \mathsf{Num_Rows}(x) = 0$$

for $1 \leq x \leq g^{177B}$.

4. The $\mathsf{Inverse}$ encodes the multiplicative inverse of $\mathsf{Remainder} - 176$.

$$(176 - \mathsf{Remainder}(x)) * ((176 - \mathsf{Remainder}(x)) * \mathsf{Inverse}(x) - 1) = 0$$

for $1 \leq x \leq g^{177B}$.

5. When $\mathsf{Remainder} = 176$ the next value must be 0.

$$(1 - (176 - \mathsf{Remainder}(x)) * \mathsf{Inverse}(x)) * (\mathsf{Remainder}(gx)) = 0$$

for $1 \leq x \leq g^{177B-1}$.

6. Any register that is unconstrained during a phase retains its previous value during that phase.

State Machine Constraints

Each row in the SHA AIR matrix corresponds to either half a compression step, a schedule word computation or an alignment step. Therefore, we can employ 2 $\mathsf{Control}$ registers to encode each of the 4 states, as outlined in Table 2.1. Hence, each of the *round constraints* and *schedule constraints* can be gated by the $\mathsf{Control}$ registers as follows:

1. We identify the semantics that the row must correspond to.
2. We negate the control registers for rows where the constraint must apply, and leave them as is for rows where the constraint does not apply, and multiply them to obtain the *constraint gate*.
3. We multiply the *contraint gate* with the actual constraint to obtain the final *gated constraint*.

For example, constraints that should be active only during the first half of the compression round will be gated as:

$$(1 - \mathsf{Control}[0](x)) * (1 - \mathsf{Control}[1](x)) * Constraint(x) = 0$$

This will cause the constraint to be inactive for all rows with a control code other than $(0, 0)$. Likewise, if the control code is $(0, 0)$ then the original constraint must hold for that row. Henceforth, when describing a constraint (or a group of constraints) we specify in italics the semantics applicable for that constraint / group of constraints.

To ensure a correct transition between the different word selectors, we employ the following constraints:

1. The schedule word selectors shift right every *first half of compression round* until the last selector is active.

$$(1 - \mathsf{W_Selector}[16](x)) * (\mathsf{W_Selector}[i](gx) - \mathsf{W_Selector}[i-1](x)) = 0 \quad \forall i \in \{1, ..., 16\}$$
$$(1 - \mathsf{W_Selector}[16](x)) * \mathsf{W_Selector}[0](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

Table 2.1: Each combination of Control registers maps to a specific semantic. This helps enforce which constraints must be active for which parts of the trace.

Control Code	Row Semantics
$(0, 0)$	First half of compression round
$(0, 1)$	Second half of compression round
$(1, 0)$	Schedule word computation
$(1, 1)$	Alignment step

2. The schedule word selectors are preserved during the *second half of the compression round* and *schedule word computation* as long as the next row is not an *alignment step*.

$$\mathsf{W_Selector}[i](gx) - \mathsf{W_Selector}[i](x) = 0 \quad \forall i \in \{0, ..., 16\}$$

for $1 \leq x \leq g^{177B-1}$.

3. The schedule word selectors reset during the *alignment step*.

$$\begin{aligned} 1 - \mathsf{W_Selector}[0](x) &= 0 \\ \mathsf{W_Selector}[i](x) &= 0 \quad \forall i \in \{1, ..., 16\} \end{aligned}$$

for $1 \leq x \leq g^{177B-1}$.

For clarity, we opt to outline **gated** variant of the constraints governing the correct transition between the different control registers:

1. The second half of the compression round always succeeds the first.

$$\begin{aligned} \prod_{i=0}^{1}(1 - \mathsf{Control}[i](x)) * \mathsf{Control}[0](gx) &= 0 \\ \prod_{i=0}^{1}(1 - \mathsf{Control}[i](x)) * (1 - \mathsf{Control}[1](gx)) &= 0 \end{aligned}$$

for $1 \leq x \leq g^{177B-1}$.

2. The schedule word computation follows the second half of the compression round when the last selector is active.

$$\begin{aligned} (1 - \mathsf{Control}[0](x)) * \mathsf{Control}[1](x) * \mathsf{W_Selector}[16](x) * (1 - \mathsf{Control}[0](gx)) &= 0 \\ (1 - \mathsf{Control}[0](x)) * \mathsf{Control}[1](x) * \mathsf{W_Selector}[16](x) * \mathsf{Control}[1](gx) &= 0 \end{aligned}$$

for $1 \leq x \leq g^{177B-1}$.

3. The first half of the compression function follows the second half when the last selector is not active.

$$(1 - \mathsf{Control}[0](x)) * \mathsf{Control}[1](x) * (1 - \mathsf{W_Selector}[16](x)) * \mathsf{Control}[0](gx) = 0$$

$$(1 - \mathsf{Control}[0](x)) * \mathsf{Control}[1](x) * (1 - \mathsf{W_Selector}[16](x)) * \mathsf{Control}[1](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

4. The first half of the compression function follows the schedule word computation.

$$\mathsf{Control}[0](x) * (1 - \mathsf{Control}[1](x)) * \mathsf{Control}[0](gx) = 0$$
$$\mathsf{Control}[0](x) * (1 - \mathsf{Control}[1](x)) * \mathsf{Control}[1](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

5. When $\mathsf{Remainder} = 176$ the next code corresponds to the alignment step.

$$(1 - (176 - \mathsf{Remainder}(x)) * \mathsf{Inverse}(x)) * (1 - \mathsf{Control}[i](gx)) = 0 \quad \forall i \in \{0, 1\}$$

for $1 \leq x \leq g^{177B-1}$.

6. The first half round always follows the alignment step.

$$\prod_{i=0}^{1} \mathsf{Control}[i](x) * \mathsf{Control}[j](gx) = 0 \quad \forall j \in \{0, 1\}$$

for $1 \leq x \leq g^{177B-1}$.

First Half Round Constraints

During the first half of the compression step, we update the e...h registers and all of the following constraints are gated by the next row having the *first half of compression round* control code.

1. The first 3 sets of Bits registers store the bitwise decomposition of $\mathsf{e}, \mathsf{f}, \mathsf{g}$.

$$\mathsf{e}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[0, i](gx) = 0$$
$$\mathsf{f}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[1, i](gx) = 0$$
$$\mathsf{g}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[2, i](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

2. Bits sets 4 and 5 store the bitwise decomposition of the output of the $\sum_1^{\{256\}}$ and Ch functions, while the last two sets act as carries. For the exact definition of the two functions, we refer the reader to [Nat15].

$$\Sigma_1^{\{256\}}[i] = \mathsf{Bits}[0, (i+6) \mod 32](gx) + \mathsf{Bits}[0, (i+11) \mod 32](gx) + \mathsf{Bits}[0, (i+25) \mod 32](gx)$$

$$Ch[i] = \mathsf{Bits}[0,i](gx) \cdot \mathsf{Bits}[1,i](gx) + (1 - \mathsf{Bits}[0,i](gx)) \cdot \mathsf{Bits}[2,i](gx)$$
$$\mathsf{Bits}[3,i](gx) + 2 \cdot \mathsf{Bits}[5,i](gx) - \Sigma_1^{\{256\}}[i] = 0$$
$$\mathsf{Bits}[4,i](gx) + 2 \cdot \mathsf{Bits}[6,i](gx) - Ch[i] = 0$$

for $1 \leq x \leq g^{177B-1}$.

3. The Bitwise_Output registers store the 32-bit equivalent of the Bits sets 4 and 5.

$$\mathsf{Bitwise_Output}[i](gx) - \sum_{k=0}^{31} 2^k \cdot \mathsf{Bits}[3+i,k](gx) = 0 \quad \forall i \in \{0,1\}$$

for $1 \leq x \leq g^{177B-1}$.

4. $\mathsf{T_1}$ and e...h registers are computed according to the [Nat15] specification.

$$W = \sum_{i=0}^{15} \mathsf{W}[i](gx) * \mathsf{W_Selector}[i](gx) + \mathsf{W}[15](gx) * \mathsf{W_Selector}[16](gx)$$
$$\mathsf{T_1}(gx) + 2^{32} \cdot \sum_{k=0}^{4} \mathsf{Carry}[k](gx) - \mathsf{h}(x) - \sum_{k'=0}^{1} \mathsf{Bitwise_Output}[k'](gx) - \mathsf{Constant}(gx) - W = 0$$
$$\mathsf{h}(gx) - \mathsf{g}(x) = 0$$
$$\mathsf{g}(gx) - \mathsf{f}(x) = 0$$
$$\mathsf{f}(gx) - \mathsf{e}(x) = 0$$
$$\mathsf{e}(gx) + 2^{32} \cdot \mathsf{Carry}[5](gx) - \mathsf{d}(x) - \mathsf{T_1}(gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

Remark 2. *Here,* Constant *stores a round constant, whose value is specified in [Nat15] according to the round index. While we do not constrain this register, we argue that allowing the adversary to input arbitrary round constants does not change the security of the AIR matrix's semantics. In particular, we argue that finding a collision in the AIR matrix is as hard as finding a collision in the standard SHA-256 function.*

Second Half Round Constraints

During the first half of the compression step, we update the a...d registers and all of the following constraints are gated by the next row having the *second half of compression round* control code.

1. The first 3 sets of Bits registers store the bitwise decomposition of $\mathsf{a}, \mathsf{b}, \mathsf{c}$.

$$\mathsf{a}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[0,i](gx) = 0$$
$$\mathsf{b}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[1,i](gx) = 0$$

$$\mathsf{c}(x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[2, i](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

2. Bits sets 4 and 5 store the bitwise decomposition of the output of the $\sum_0^{\{256\}}$ and Maj functions, while the last two sets act as carries.

$$\Sigma_0^{\{256\}}[i] = \mathsf{Bits}[0, (i+2) \mod 32](gx) + \mathsf{Bits}[0, (i+13) \mod 32](gx) + \mathsf{Bits}[0, (i+22) \mod 32](gx)$$
$$Maj[i] = \mathsf{Bits}[0, i](gx) \cdot \mathsf{Bits}[1, i](gx) + \mathsf{Bits}[0, i](gx) \cdot \mathsf{Bits}[2, i](gx) + \mathsf{Bits}[1, i](gx) \cdot \mathsf{Bits}[2, i](gx)$$
$$\mathsf{Bits}[3, i](gx) + 2 \cdot \mathsf{Bits}[5, i](gx) - \Sigma_0^{\{256\}}[i] = 0$$
$$\mathsf{Bits}[4, i](gx) + 2 \cdot \mathsf{Bits}[6, i](gx) - Maj[i] = 0$$

for $1 \leq x \leq g^{177B-1}$.

3. The Bitwise_Output registers store the 32-bit equivalent of the Bits sets 4 and 5.

$$\mathsf{Bitwise_Output}[i](gx) - \sum_{k=0}^{31} 2^k \cdot \mathsf{Bits}[3+i, k](gx) = 0 \quad \forall i \in \{0, 1\}$$

for $1 \leq x \leq g^{177B-1}$.

4. T_2 and a...d registers are computed according to the [Nat15] specification.

$$\mathsf{T}_2(gx) + 2^{32} \cdot \mathsf{Carry}[0](gx) - \sum_{i=0}^{1} \mathsf{Bitwise_Output}[i](gx) = 0$$
$$\mathsf{d}(gx) - \mathsf{c}(x) = 0$$
$$\mathsf{c}(gx) - \mathsf{b}(x) = 0$$
$$\mathsf{b}(gx) - \mathsf{a}(x) = 0$$
$$\mathsf{a}(gx) + 2^{32} \cdot \mathsf{Carry}[1](gx) - \mathsf{T}_1(gx) - \mathsf{T}_2(gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

Schedule Word Update Constraints

During this phase, we enforce a correct computation of the next word in the message schedule and gate the following constraints by the next row having the *schedule word computation* control code.

1. The first 15 schedule registers shift left by 1 position.

$$\mathsf{W}[i](gx) - \mathsf{W}[i+1](x) = 0 \quad \forall i = \overline{0..14}$$

for $1 \leq x \leq g^{177B-1}$.

2. The first two Bits sets hold the bitwise decomposition of W_{t-2} and W_{t-15}.

$$\mathsf{W}[13](x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[0,i](gx) = 0$$
$$\mathsf{W}[0](x) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[1,i](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

3. Bits sets 3 and 4 store the output of $\sigma_1^{\{256\}}(W_{t-2})$ and $\sigma_0^{\{256\}}(W_{t-15})$ respectively, while Bits sets 5 and 6 act as carries.

$$\sigma_1^{\{256\}}[i] = \mathsf{Bits}[0,(i+17) \mod 32](gx) + \mathsf{Bits}[0,(i+19) \mod 32](gx) + \mathsf{Bits}[0,i+10](gx)$$
$$\sigma_0^{\{256\}}[i] = \mathsf{Bits}[1,(i+7) \mod 32](gx) + \mathsf{Bits}[1,(i+18) \mod 32](gx) + \mathsf{Bits}[1,i+3](gx)$$
$$\mathsf{Bits}[2,i](gx) + 2 \cdot \mathsf{Bits}[4,i](gx) - \sigma_1^{\{256\}}[i] = 0$$
$$\mathsf{Bits}[3,i](gx) + 2 \cdot \mathsf{Bits}[5,i](gx) - \sigma_0^{\{256\}}[i] = 0$$

for $1 \leq x \leq g^{177B-1}$.

Remark 3. *Since the last terms of σ_0, σ_1 will have out of bounds indices as $i \to 31$, we force $\mathsf{Bits}[i,j] = 0 \; \forall j \geq 32$. Consequently, $\mathsf{Bits}[i,j+k]$ corresponds to a bitwise right-shift of k. Furthermore, bitwise operations can be easily encoded as operations in the binary field. For example $a \oplus b = (a+b) \mod 2$.*

4. The $\mathsf{Bitwise_Output}$ registers store the 32-bit equivalent of the Bits sets 3 and 4.

$$\mathsf{Bitwise_Output}[i](gx) - \sum_{j=0}^{31} 2^j \cdot \mathsf{Bits}[2+i,j](gx) = 0 \quad \forall i = \overline{0..1}$$

for $1 \leq x \leq g^{177B-1}$.

5. The next schedule word is computed according to the [Nat15] specification.

$$\mathsf{W}[15](gx) + 2^{32} \cdot \sum_{k=0}^{1} \mathsf{Carry}[k](gx) - \sum_{k'=0}^{1} \mathsf{Bitwise_Output}[k'](gx) - \mathsf{W}[9](x) - \mathsf{W}[0](x) = 0$$

for $1 \leq x \leq g^{177B-1}$.

6. The last W register is decomposed into 32-bits which are stored in the last Bits set.

$$\mathsf{W}[15](gx) - \sum_{i=0}^{31} 2^i \cdot \mathsf{Bits}[6,i](gx) = 0$$

for $1 \leq x \leq g^{177B-1}$.

Remark 4. *The role of this decomposition is to attest that the next word is indeed 32-bit. This, combined with the fact that all W registers are eventually decomposed ensure that the prover did not introduce input blocks larger than 32 bits.*

Alignment Step Constraints

During the alignment step we allow the prover to insert new input blocks in the W registers, and aggregate the current a...h registers into the output hash. Therefore, all of the following constraints are gated by the next row having the *allignment step* control code.

1. The Hash registers are correctly updated using the current a...h registers.

$$\begin{aligned}
\mathsf{H}[0](gx) + 2^{32} \cdot \sum_{k=0}^{1} \mathsf{Carry}[k](gx) - \mathsf{H}[0](x) - \mathsf{a}(x) = 0 \\
\mathsf{H}[1](gx) + 2^{32} \cdot \sum_{k=2}^{3} \mathsf{Carry}[k](gx) - \mathsf{H}[1](x) - \mathsf{b}(x) = 0 \\
\dots \\
\mathsf{H}[7](gx) + 2^{32} \cdot \sum_{k=14}^{15} \mathsf{Carry}[k](gx) - \mathsf{H}[7](x) - \mathsf{h}(x) = 0
\end{aligned}$$

   for $1 \leq x \leq g^{177B-1}$.

2. The a...h registers are equal to the previous hash digest.

$$\begin{aligned}
\mathsf{a}(gx) - \mathsf{H}[0](x) = 0 \\
\dots \\
\mathsf{h}(gx) - \mathsf{H}[7](x) = 0
\end{aligned}$$

   for $1 \leq x \leq g^{177B-1}$.

3. All registers that are unaffected by a constraint during this phase are zeroed, with the exception of the W registers, which are left unconstrained to allow for more prover input.

Note that the first row of the AIR matrix is actually an alignment step. However, the above constraints will not apply due to the gate being inactive. Therefore, in this AIR matrix we employ assertions (i.e., constraints that apply to only 1 row), while in AIR matrices that include the SHA-256 AIR as a sub-matrix a dedicated indicator will mark the start of the sub-matrix. Namely, to ensure a semantically correct hash round we employ the following assertions:

1. The a...h and the H registers are initialized to the constants defined in [Nat15].

$$\begin{aligned}
\mathsf{a}(0) - \mathsf{H_0} = 0 \\
\dots \\
\mathsf{h}(0) - \mathsf{H_1} = 0 \\
\mathsf{H}[i](0) - \mathsf{H_i} = 0 \quad \forall i \in \{0, ..., 7\}
\end{aligned}$$

2. The Num_Rows register starts at 0.

$$\mathsf{Num_Rows}(0) = 0$$

3. All registers that are unaffected by a general constraint or one of the previous constraints, are zero.

Bibliography

[Nat15] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication FIPS PUB 180-4. National Institute of Standards and Technology, Aug. 2015. URL: https://doi.org/10.6028/NIST.FIPS.180-4 (visited on 01/23/2026).